Documentacion tecnica de Syslab Scraper HHR

Actualizado: 2026-05-06

Este documento describe como funciona **Syslab Scraper**, el microservicio local que permite a HHR consultar examenes de laboratorio desde el portal interno Syslab (`http://10.4.69.90/syslab/`).

El punto clave operativo es que Syslab vive en una red privada del hospital. Por eso el frontend HHR desplegado en Netlify no puede llegar directamente a `10.4.69.90` ; necesita pasar por un puente local que este encendido dentro de la red hospitalaria.

Arquitectura actual

```
HHR local
  -> http://localhost:3000
  -> Syslab Scraper local
  -> http://10.4.69.90/syslab/

HHR desplegado en Netlify
  -> /.netlify/functions/syslab-proxy
  -> SYSLAB_PROXY_URL publico
  -> tunel ngrok/cloudflared abierto desde el Mac local
  -> http://localhost:3000
  -> Syslab Scraper local
  -> http://10.4.69.90/syslab/
```

En local, HHR conversa con `localhost:3000` usando `VITE_SYSLAB_API_URL=http://localhost:3000` .

En produccion, HHR conversa con la Function `syslab-proxy` de Netlify. Esa Function necesita una variable de entorno `SYSLAB_PROXY_URL` con una URL publica que apunte al scraper local por medio de un tunel. Si el tunel esta apagado, Netlify responde `503` y el boton **Lab** queda deshabilitado.

Componentes principales

- `server.js` : arranca Express, sirve `public/` , expone `/health` y monta las rutas `/api` .
- `config/env.js` : centraliza `PORT` , `SYSLAB_BASE_URL` , `SYSLAB_USER` y `SYSLAB_PASS` .
- `routes/api.js` : define los endpoints de laboratorio.
- `controllers/examController.js` : valida parametros HTTP y delega en el orquestador.
- `services/browserManager.js` : inicializa Chromium con Playwright y realiza login en Syslab.
- `services/scraperOrchestrator.js` : coordina busqueda de ordenes, descarga/captura de PDFs y extraccion de resultados.
- `utils/reportParser.js` : transforma texto crudo de PDF en resultados estructurados.
- `logs/` : contiene errores, HTML de diagnostico y textos crudos extraidos de PDFs.

Variables de entorno

El scraper lee `.env` desde la raiz del proyecto.

```
PORT=3000
SYSLAB_BASE_URL=http://10.4.69.90/syslab/
SYSLAB_USER=<usuario_syslab>
SYSLAB_PASS=<clave_syslab>
```

El codigo tiene valores por defecto para desarrollo, pero en operacion real conviene mantenerlos explicitamente en `.env`.

En HHR/Netlify, la variable relevante es:

```
SYSLAB_PROXY_URL=https://<tunel-publico>
```

Ejemplo actual usado por Netlify:

```
SYSLAB_PROXY_URL=https://hyperkalemic-transthoracic-lecia.ngrok-free.dev
```

Esa URL no la levanta Netlify. La levanta el Mac local ejecutando ngrok o cloudflared hacia el puerto `3000`.

Endpoints

`GET /health`

Verifica que el microservicio Express esta encendido.

Respuesta esperada:

```
{
  "status": "ok",
  "service": "syslab-scraper",
  "version": "1.0.0",
  "timestamp": "2026-05-06T20:12:22.022Z"
}
```

`GET /api/exams?rut=<rut>`

Busca las ordenes de laboratorio asociadas a un RUT.

Flujo:

1. Recibe el RUT desde HHR.
2. Inicia Chromium headless con Playwright.
3. Abre `SYSLAB_BASE_URL`.
4. Realiza login con `SYSLAB_USER` y `SYSLAB_PASS`.
5. Navega a la busqueda por RUT.
6. Normaliza el RUT a cuerpo numerico, sin puntos, guion ni digito verificador.
7. Extrae filas de `table.narrow tr`.
8. Devuelve ordenes con `id`, `link`, `date`, `time`, `patientName`, `origin` y lista de examenes.

Ejemplo:

```
curl "http://localhost:3000/api/exams?rut=10096004"
```

`POST /api/exams/details`

Recibe una lista de links de ordenes y devuelve resultados estructurados.

Entrada:

```
{
  "links": [
    "http://10.4.69.90/syslab/detalleexamenes.php?id=43092478&user=..."
  ]
}
```

Flujo:

1. Abre una sesion autenticada en Syslab.
2. Procesa links con concurrencia controlada.
3. Captura PDF mediante listeners de red, evento de descarga o links embebidos.
4. Si no encuentra PDF nativo, genera un PDF desde el HTML renderizado como ultimo fallback.
5. Extrae texto con `pdf-parse`.
6. Guarda texto crudo en `logs/raw_extract_*.txt`.
7. Parsea variables con `utils/reportParser.js`.

`GET /api/exams/pdf?link=<url>`

Devuelve el PDF de una orden para visualizacion inline en HHR.

El servicio reautentica contra Syslab, navega al link y captura el PDF por respuesta de red, descarga nativa o renderizado HTML como fallback.

Flujo local de operacion

Terminal 1: levantar Syslab Scraper.

```
cd "/Users/daniel/Documents/scraper LAB"
node server.js
```

Verificar:

```
curl "http://localhost:3000/health"
curl "http://localhost:3000/api/exams?rut=10096004"
```

Terminal 2: levantar HHR local en un puerto distinto de `3000`.

```
cd "/Users/daniel/Documents/HHR 2026 tracker version MacBookAir"
VITE_SYSLAB_ENABLE_DIRECT_LOCAL=true npm run dev -- --host 0.0.0.0 --port 3020
```

HHR local queda en:

```
http://localhost:3020/
```

Flujo para Netlify/despliegue publico

Para que la pagina desplegada en Netlify pueda usar Lab:

1. El scraper debe estar corriendo localmente en `localhost:3000`.
2. El Mac debe estar dentro de la red hospitalaria o con acceso real a `10.4.69.90`.
3. Debe existir un tunel publico activo hacia `localhost:3000`.
4. Netlify debe tener `SYSLAB_PROXY_URL` apuntando a esa URL publica.

Ejemplo con ngrok:

```
npm run tunnel:ngrok
```

Verificar el tunel:

```
curl "https://hyperkalemic-transthoracic-lecia.ngrok-free.dev/health"
curl "https://hyperkalemic-transthoracic-lecia.ngrok-free.dev/api/exams?rut=10096004"
```

Si el endpoint publico de ngrok muestra `ERR_NGROK_3200`, el tunel esta offline. En ese estado Netlify no puede usar Lab aunque el codigo de HHR este correcto.

Diagnostico rapido

El boton Lab aparece deshabilitado en Netlify

Causa mas frecuente: `/.netlify/functions/syslab-proxy?action=health` responde `503`.

Checklist:

```
npm run check:syslab
curl "http://localhost:3000/health"
curl "https://<SYSLAB_PROXY_URL>/health"
netlify env:get SYSLAB_PROXY_URL
```

Interpretacion:

- Local `/health` falla: `node server.js` no esta corriendo.
- Local `/health` funciona, pero URL publica falla: el tunel esta apagado o apunta a otro puerto.
- URL publica funciona, pero Netlify falla: revisar `SYSLAB_PROXY_URL`, auth/rol o logs de Netlify Function.

`npm run check:syslab` ejecuta una verificacion resumida del scraper local y del tunel publico. Tambien muestra el acceso directo a Syslab como advertencia orientativa, porque el flujo real usa Playwright y puede comportarse distinto a `curl`.

Error `page.goto: net::ERR_ADDRESS_UNREACHABLE`

Playwright no puede abrir `http://10.4.69.90/syslab/`.

Checklist:

```
curl -I --max-time 5 "http://10.4.69.90/syslab/"
ping 10.4.69.90
```

Posibles causas:

- El Mac no esta en la red hospitalaria correcta.
- El servidor Syslab esta apagado o cambio de IP.
- Hay bloqueo de red, VLAN o firewall.
- El cable/adaptador/interfaz no tiene ruta valida.

`localhost` funciona, pero Netlify no

Esto es esperable si no hay tunel. Netlify no puede acceder al `localhost` del Mac. Netlify solo puede llamar una URL publica configurada como `SYSLAB_PROXY_URL`.

Logs utiles

- `logs/combined.log`: solicitudes y eventos generales.
- `logs/error.log`: fallos de login, navegacion y scraping.
- `logs/debug_post_aceptar_*.html`: HTML posterior a la busqueda por RUT.
- `logs/raw_extract_*.txt`: texto crudo extraido de PDFs.
- `logs/login_error_*.png`: captura cuando falla el login.

Pruebas

```
npm test
```

Pruebas relevantes:

- `tests/scraperExamListExtraction.test.js`: valida extraccion de filas de examenes.
- `tests/scraperOrchestratorConcurrency.test.js`: valida helpers de concurrencia.

Limites y riesgos operativos

- El scraper depende del HTML real de Syslab; cambios de DOM pueden romper selectores.
- El portal Syslab no expone API publica, por lo que Playwright actua como un usuario automatizado.
- El tunel publico debe quedar encendido para produccion Netlify.
- Un tunel ngrok gratuito puede quedar offline o cambiar comportamiento si no se mantiene activo.
- No conviene disparar consultas masivas desde el navegador clinico durante atencion; las consultas deben ser acotadas o programadas.

Resumen ejecutivo

Syslab Scraper funciona como puente entre HHR y un sistema interno sin API. En local, HHR usa `localhost:3000`. En Netlify, HHR usa una Function serverless que llama a `SYSLAB_PROXY_URL`. Esa URL debe ser un tunel activo desde un computador dentro de la red hospitalaria hacia el scraper local. Si el tunel se apaga, el codigo puede estar perfecto y aun asi Lab aparecera deshabilitado en produccion.